

LC 127: Word Ladder

Google

Tekion

Zscaler

Adobe

Problem statement & Constraints

given `beginword` → `endword`

it's possible both are not in dictionary, we need short transformation sequence length.

at one time only one character can change.

Example

start = hit

end = cog

dictionary = ["hot", "dot", "dog", "lot", "log", "cog"]

hit (1) → hot (2) → dot (3) → dog (4) → cog (5)

O/p = 5

Graph Representation

- # every word is a node
- # 2 words have an edge if they differ by one

```
    hot
   /  \
 dot   lot
  |
 dog
  |
```

Algorithm Choice

Shortest path in an unweighted graph → **BFS**

Issue: Graph not given, we need to generate neighbours.

cog

Neighbour Generation Logic

i.e. for **hit** →

```
ait, bit, cit ...  
hat, hbt, hct ...  
hia, hib, hic ...
```

→ *only those in dictionary are visited*

Time Complexity: $O(N \times L \times 26)$

C++ Implementation

```
class Solution {  
public:  
    int ladderLength(string beginWord, string endWord, vector<string>& wordList) {  
  
        unordered_set<string> st; //for faster lookups  
        unordered_map<string,int> seen;  
        queue<pair<string,int>> q;  
  
        //basically bfs krenge for every string , ham ek character change krenge and dek  
        //ki wo dictionary me he ya nahi  
  
        for(string s:wordList){  
            st.insert(s);  
        }  
  
        if(st.find(endWord)==st.end()) return 0; //he hi nhi dictionary me  
  
        q.push({beginWord,1});  
        seen[beginWord]=1;  
  
        while(!q.empty()){  
            string currs=q.front().first;  
            int currd=q.front().second;  
  
            q.pop();  
  
            if(currs==endWord) return currd;  
        }  
    }  
};
```

```
for(int i=0;i<currs.size();i++){ //AB he to AB ka BB CB ; AA AB AC sab bana

    char orig=currs[i];
    for(char c='a';c<='z';c++){
        if(c==orig) continue;
        currs[i]=c;
        if(st.find(currs)!=st.end() && seen.find(currs)==seen.end()){
            seen[currs]=1;
            q.push({currs,currd+1});
        }
    }

    currs[i]=orig; //restore original string before going to flip some other

}

return 0;

}

};
```